Machine Learning Project

Milestone 1: Data and Prototype

Paper: Do-PFN: In-Context Learning for Causal Effect Estimation, Robertson, Reuter et al., NeurIPS 2025, arXiv 2506.06039

Goal

The goal of this milestone is to understand the paper, become familiar with the data and get a feel for the task: given an observational dataset of treatment, covariates and outcome, predict the outcome under an intervention on the treatment, and from it the conditional average treatment effect (CATE). The model is pre-trained on data simulated from synthetic structural causal models (SCMs). For this, you build a data loader, a visualization tool and a working prototype, starting from `Do-PFN-prior`. Note that it only generates observational data and has no designated treatment; adding a binary treatment and sampling interventional data is part of your work.

The prototype does not have to match the results of the paper, but pre-training has to converge: the loss should decrease and the trained model should beat the simple baselines on data from the prior. If you are unsure about the next steps, discuss them with your project supervisor.

Implementation (1 point)

- **Data loading:** Write a function

```
def load_data(dataset='prior',
              transformation=None,
              n_train=None, n_test=None, ...):
```

  A single sample is a whole dataset. For `dataset='prior'`, it samples an SCM, an observational dataset and interventional query points from the same SCM; for the case studies from the Do-PFN repository (one value per case study, e.g. `dataset='observed_confounder'`), it reads them from disk. It applies `transformation` to every dataset and returns an iterable (lazy loading) that yields tuples $(X^{ob}, y^{ob}, X^{in}, y^{in})$, with the treatment in the first column of X, together with the observational outcomes of the query points (needed to train Dont-PFN in Milestone 2) and the ground-truth CATE.
- **Data on the cluster:** Store the data in `/home/space/datasets/<dataset>` and make it readable (`chmod -R a+r /home/space/datasets/<dataset>`).
- **Visualization:** Write a function

```
def show(x, outfile=None, ...):
```

  which takes a list of data points and outputs a visualization of the data as an image, optionally saved to `outfile` (e.g. the causal graph next to observational and interventional data). Keep it extensible, as you might want to add overlays later, and make sure the outputs are visually pleasing (they ought to go in the report).
- **Model prototype:** Write a model class

```
class Model(torch.nn.Module):
    def __init__(self, ...):
        super().__init__()
        [...]
```

```
    def forward(self, X, y_ob):
        [...]
```

The `__init__` function takes hyperparameters and sets up the trainable parameters. The method `forward` takes a batch of tables `X` (observational rows followed by interventional query rows) and the observational outcomes `y_ob`, and returns a predictive distribution over the interventional outcomes (e.g. a bar distribution as in the paper). It should be differentiable, so use only pytorch functions and be careful with conditionals and loops. Also implement the loss function and a pre-training loop on data from your prior (e.g. based on the training loop in `Do-PFN-prior`, which is adapted from TFM-Playground) and train the model until the loss converges.

- **Scikit-learn interface:** Wrap the model in a class with a scikit-learn-like interface, similar to `DoPFNRegressor` in the Do-PFN repository:

```
class Regressor:
    def __init__(self, model, ...): [...]
    def fit(self, X_ob, y_ob): [...]
    def predict(self, X): [...]
    def predict_cate(self, X): [...]
```

`fit` only stores the observational data; `predict` returns the predicted outcome under the treatment given in the first column of `X`, and `predict_cate` the difference between treatment 1 and 0.

Report (9 points)

Dataset overview (4 points)

The points below are guidelines, not a checklist to tick off.

- **Exploration:** What kinds of SCMs and datasets does the prior generate, and how different are observational and interventional data? What do the case studies look like? Show examples using `show`.
- **Data quality and splits:** Check for problems that could distort the evaluation, e.g. degenerate SCMs, treatment imbalance, lack of overlap between treated and untreated, or overlap between training and test data.
- **Preprocessing and representation:** Describe the preprocessing and encoding, why you chose them, and which `transformation` you pass to `load_data`. Can the model handle datasets of different size or shape?
- **Questions:** In which case studies is the causal effect identifiable? What kind of answers could a machine learning model provide about the data, and what would a human typically ask?

Baseline and evaluation (3 points)

- **Baseline:** Describe simple baselines (e.g. a regression model on the observational data queried with $t = 0$ and $t = 1$, or a difference of group means) and the first approach from your prototype. Explain why you chose them.
- **Metrics:** Which metrics are meaningful, and why? Why can you not evaluate causal effects on purely observational real-world data?
- **Results:** Show the training and validation loss curves of your prototype and report the results

of your baselines and the trained prototype, if possible broken down by case study. Where do they fail?
- **Overfitting:** Is overfitting an issue? How do you check for it, and how do you avoid it?
- **Non-technical view:** Can you say something non-technical about what makes the task easy or hard?

Discussion (2 points)

- **Method:** Briefly explain the method of the paper: what does the model approximate, and how do the architecture and the prior achieve it?
- **Challenges:** Why is the task not trivial? Why does a regression model fitted on observational data not answer an interventional question?
- **Summary:** What does (not) work, and why (not)?
- **Outlook:** What do you think is possible on this data set? What would you like to try next?

Literature and Resources

- **Paper:** Robertson, Reuter, Guo, Hollmann, Hutter, Schölkopf: Do-PFN: In-Context Learning for Causal Effect Estimation. NeurIPS 2025, arXiv:2506.06039. Code of the authors (pre-trained model, prior and case studies): `https://github.com/jr2021/Do-PFN`
- **Starting point:** Do-PFN-prior, a simple configurable SCM prior compatible with NanoTabPFN: `https://github.com/oossen/Do-PFN-prior`
- **nanoTabPFN:** Pfefferle et al., a minimal TabPFN with a scikit-learn-like interface: `https://github.com/automl/nanoTabPFN`
- **TFM-Playground** (NanoTabPFN training loop): `https://github.com/automl/TFM-Playground`
- **PFNs:** Müller et al.: Transformers Can Do Bayesian Inference. ICLR 2022
- **Causality:** Peters, Janzing, Schölkopf: Elements of Causal Inference. MIT Press, 2017
- PyTorch documentation: `https://pytorch.org/docs/stable/index.html`
- scikit-learn documentation: `https://scikit-learn.org/stable/documentation.html`

How to submit

- **Report:** The report must be submitted via ISIS. Do not include code in your report. Reports that are longer than ten pages will not be accepted. Use the LaTeX template provided on the ISIS course page.
- **Code:** Create a repository on GitHub for your project and share it with your supervisor (GitHub handle `marvinsxtr`). Tag the commit you submit with `ms1`. The code has to be executable on the cluster. Do not submit Jupyter notebooks.